

P1: Building the Ultimate Tic-Tac-Toe Engine in Python

This is the first practitioner post in the series. The theory essays explain why things work; the practitioner posts show how to build them. You need Python 3.9+ and NumPy. All code in this post is self-contained — you can paste it directly into a notebook or script. The full runnable notebook is available on [Google Colab](#).

Before we can train any AI, we need a game engine: something that knows the rules, generates legal moves, applies them, and tells us when the game is over. This post builds that engine from scratch. By the end, you will have a fully functional UTTT implementation and an understanding of how game state is encoded as a tensor for neural network input.

1. Representing the Board

The first design decision is how to represent the game state. We want something that is easy to reason about, efficient to copy (we will be making millions of copies during search), and expressive enough to capture everything the rules require.

A dataclass with NumPy arrays works well:

```
import numpy as np
from dataclasses import dataclass, field

@dataclass
class GameState:
    cells: np.ndarray = field(
        default_factory=lambda: np.zeros((9, 9), dtype=np.int8)
    )
    sub_board_results: np.ndarray = field(
        default_factory=lambda: np.zeros(9, dtype=np.int8)
    )
    active_sub_board: int = -1
    current_player: int = 1
    move_count: int = 0
    is_terminal: bool = False
    winner: object = None
```

Breaking this down field by field:

cells — shape (9, 9): The 81 squares of the board, indexed as `cells[sub_board, cell]`. Values are +1 (X), -1 (O), or 0 (empty). Sub-boards and cells are both indexed 0-8, left-to-right then top-to-bottom within each 3×3 grid.

sub_board_results — shape (9,): The status of each of the 9 local boards. Values are +1 (X won), -1 (O won), 2 (draw), or 0 (still in play).

active_sub_board: Which local board the current player must play in. -1 means free choice — the player may play anywhere.

current_player: +1 for X, -1 for O. This sign convention makes win detection and symmetry handling elegant.

is_terminal and winner: Terminal states end the game. winner is +1 (X wins), -1 (O wins), or 0 (draw).

Why immutability matters

We make the game state immutable by always creating a copy before applying a move:

```
def copy(self):
    return GameState(
        cells=self.cells.copy(),
        sub_board_results=self.sub_board_results.copy(),
        active_sub_board=self.active_sub_board,
        current_player=self.current_player,
        move_count=self.move_count,
        is_terminal=self.is_terminal,
        winner=self.winner,
    )
```

Immutable states mean that the tree search can hold references to many board positions simultaneously without any of them corrupting each other. Mutable state is a source of hard-to-find bugs in game engines.

2. Move Encoding

UTTT has 81 possible squares. Rather than representing a move as a (sub_board, cell) pair, we encode each move as a single integer from 0 to 80:

```
def encode_move(sub_board: int, cell: int) -> int:
    return sub_board * 9 + cell

def decode_move(move_idx: int) -> tuple[int, int]:
    return move_idx // 9, move_idx % 9
```

This flat encoding is convenient for the neural network’s policy head, which outputs a probability distribution over all 81 moves. Sub-board 0, cell 0 is move 0; sub-board 8, cell 8 is move 80.

The spatial position of a cell in the 9×9 grid (for visualisation) is:

```
sub_board, cell = decode_move(move_idx)
row = (sub_board // 3) * 3 + (cell // 3)
col = (sub_board % 3) * 3 + (cell % 3)
```

3. Generating Legal Moves

Legal move generation is the core of the engine. It implements the “send your opponent” rule:

```
def get_legal_moves(state: GameState) -> list[int]:
    if state.is_terminal:
        return []
    moves = []
    if state.active_sub_board == -1:
        # Free choice: any empty cell in any undecided sub-board
        for sb in range(9):
            if state.sub_board_results[sb] != 0:
                continue
            for cell in range(9):
                if state.cells[sb, cell] == 0:
                    moves.append(encode_move(sb, cell))
    else:
        sb = state.active_sub_board
        if state.sub_board_results[sb] == 0:
            # Normal case: play in the required sub-board
            for cell in range(9):
                if state.cells[sb, cell] == 0:
```

```

        moves.append(encode_move(sb, cell))
    if not moves:
        # The required sub-board is full or already decided:
        # fall back to free choice
        for sb2 in range(9):
            if state.sub_board_results[sb2] != 0:
                continue
            for cell in range(9):
                if state.cells[sb2, cell] == 0:
                    moves.append(encode_move(sb2, cell))

    return moves

```

The key logic: if `active_sub_board` is -1, any empty cell on any undecided board is legal. Otherwise, the current player must play in `active_sub_board` — unless that board is already decided, in which case they get free choice.

For the neural network, we also need a binary mask over all 81 moves:

```

def get_legal_move_mask(state: GameState) -> np.ndarray:
    mask = np.zeros(81, dtype=np.float32)
    for move in get_legal_moves(state):
        mask[move] = 1.0
    return mask

```

This mask is multiplied with the policy head output during search to zero out illegal moves before the softmax.

4. Win Detection

Win detection runs at two levels: local (did someone win this sub-board?) and global (did someone win three sub-boards in a row?).

```

WIN_LINES = [
    (0,1,2), (3,4,5), (6,7,8),    # rows
    (0,3,6), (1,4,7), (2,5,8),    # columns
    (0,4,8), (2,4,6),              # diagonals
]

def check_sub_board_winner(cells_for_sb: np.ndarray) -> int:
    """Returns +1, -1, 2 (draw), or 0 (ongoing)."""

```

```

for a, b, c in WIN_LINES:
    if cells_for_sb[a] != 0 and cells_for_sb[a] == cells_for_sb[b] == cells_for_sb[c]:
        return int(cells_for_sb[a])
if np.all(cells_for_sb != 0):
    return 2 # draw
return 0 # still in play

def check_meta_winner(sub_board_results: np.ndarray) -> int:
    """Returns +1, -1, 2 (draw), or 0 (ongoing) for the global board."""
    for a, b, c in WIN_LINES:
        if (sub_board_results[a] in (1, -1) and
            sub_board_results[a] == sub_board_results[b] == sub_board_results[c]):
            return int(sub_board_results[a])
    if np.all(sub_board_results != 0):
        return 2
    return 0

```

5. Applying a Move

`apply_move` is the heart of the engine. It takes a state and a move index, and returns a new state:

```

def apply_move(state: GameState, move_idx: int) -> GameState:
    sb, cell = decode_move(move_idx)
    s = state.copy()

    # Place the mark
    s.cells[sb, cell] = state.current_player

    # Update sub-board result if it just ended
    if s.sub_board_results[sb] == 0:
        result = check_sub_board_winner(s.cells[sb])
        if result != 0:
            s.sub_board_results[sb] = result

    # Check if the global game is over
    meta = check_meta_winner(s.sub_board_results)

```

```

if meta != 0:
    s.is_terminal = True
    s.winner = 0 if meta == 2 else meta

# The cell played determines where the opponent must go next
# If that sub-board is already decided, give free choice
s.active_sub_board = -1 if s.sub_board_results[cell] != 0 else cell

# Switch player and increment move count
s.current_player = -state.current_player
s.move_count = state.move_count + 1

return s

```

The line `s.active_sub_board = -1 if s.sub_board_results[cell] != 0 else cell` is the entire “send your opponent” rule in one expression. The cell index within the current sub-board (cell) determines the next active sub-board. If `sub_board_results[cell]` is non-zero (that sub-board is decided), we set `active_sub_board = -1` to grant free choice.

6. Encoding State as a Tensor

The game state needs to be converted to a tensor for the neural network. We use 7 channels, each a 9×9 binary grid, always expressed from the perspective of the current player (so the network always sees “my pieces” in channel 0, regardless of whether the current player is X or O):

```

def encode_state(state: GameState) -> np.ndarray:
    """Returns shape (7, 9, 9) float32 tensor."""
    p = state.current_player # +1 or -1
    tensor = np.zeros((7, 9, 9), dtype=np.float32)

    for sb in range(9):
        rb, cb = sb // 3, sb % 3
        result = state.sub_board_results[sb]

        for cell in range(9):
            r = rb * 3 + cell // 3

```

```

c = cb * 3 + cell % 3
v = state.cells[sb, cell]

# Channel 0: current player's pieces
if v == p:
    tensor[0, r, c] = 1.0
# Channel 1: opponent's pieces
elif v == -p:
    tensor[1, r, c] = 1.0

# Channels 2-4: sub-board outcomes (current player won, opponent won, draw)
if result == p:
    tensor[2, rb*3:rb*3+3, cb*3:cb*3+3] = 1.0
elif result == -p:
    tensor[3, rb*3:rb*3+3, cb*3:cb*3+3] = 1.0
elif result == 2:
    tensor[4, rb*3:rb*3+3, cb*3:cb*3+3] = 1.0

# Channel 5: active sub-board mask
is_active = (state.active_sub_board == -1 or state.active_sub_board == sb)
if is_active and result == 0:
    tensor[5, rb*3:rb*3+3, cb*3:cb*3+3] = 1.0

# Channel 6: current player indicator (all 1s for current player, 0s for opponent)
tensor[6, :, :] = 1.0 if p == 1 else 0.0

return tensor

```

The 7 channels encode different types of information:

- **Ch 0** — my pieces (current player's marks)
- **Ch 1** — opponent's pieces
- **Ch 2** — sub-boards I have won (filled with 1s across the full 3×3 region)
- **Ch 3** — sub-boards opponent has won
- **Ch 4** — drawn sub-boards
- **Ch 5** — currently active sub-board(s): where I am allowed to play
- **Ch 6** — whose turn it is (a global binary flag, constant across the grid)

The player-relative encoding (channels 0 and 1 flip depending on whose turn it is) means the network learns symmetrically — it always sees the board from the perspective of the player to move. Without this, the network would need to learn separate strategies for X and O.

7. Putting It Together: A Random Game

Here is a complete random-agent loop using everything we have built:

```
import random

def play_random_game(seed=None):
    if seed is not None:
        random.seed(seed)
    state = GameState()
    move_history = []

    while not state.is_terminal:
        legal = get_legal_moves(state)
        move = random.choice(legal)
        move_history.append(move)
        state = apply_move(state, move)

    return state, move_history

state, history = play_random_game(seed=42)
print(f"Game ended after {state.move_count} moves")
print(f"Winner: {'X' if state.winner == 1 else 'O' if state.winner == -1 else 'Draw'}")
print(f"Legal moves at each step ranged from {min(len(get_legal_moves(GameState())))} for
```

With seed 42, the game lasts 42 moves and X wins. Running 1000 random games gives a sense of the distribution:

```
results = {'X': 0, 'O': 0, 'Draw': 0}
lengths = []

for seed in range(1000):
    s, h = play_random_game(seed=seed)
    lengths.append(s.move_count)
```



```

    key = 'X' if s.winner == 1 else 'O' if s.winner == -1 else 'Draw'
    results[key] += 1

print(f"Results: {results}")
print(f"Avg game length: {sum(lengths)/len(lengths):.1f} moves")
print(f"Min/Max: {min(lengths)}/{max(lengths)}")

```

Typical output:

Results: {'X': 498, 'O': 487, 'Draw': 15}

Avg game length: 39.4 moves

Min/Max: 17/81

Random play is nearly symmetric between X and O (first-mover advantage is small), games typically last around 40 moves, and draws are rare. This baseline is useful: any trained agent should dramatically outperform random play, and any evaluation should use both colours.

8. What We Have

At this point we have a complete, self-contained UT TT game engine:

- **GameState** — immutable board representation
- **encode_move / decode_move** — flat integer move encoding
- **get_legal_moves** — full rule-compliant move generation
- **get_legal_move_mask** — binary mask for the neural network
- **apply_move** — state transition function
- **check_sub_board_winner / check_meta_winner** — win detection
- **encode_state** — 7-channel tensor encoding

This is everything the MCTS and training loop need from the game engine. P2 will wire up simple RL agents (a tabular Q-learner and a neural Q-network) against this engine, which will make the limitations of both immediately apparent and motivate the AlphaZero approach.

Theory: [T1 — What is Ultimate Tic-Tac-Toe?](#) covers the rules and strategic structure of the game. [T2 — The Reinforcement Learning Landscape](#) covers the learning algorithms that P2 will implement.

Full notebook: [Notebook 1 — The UTTT Game Engine](#) — runs all the code in this post with additional tests, an ASCII board visualiser, and a random-agent tournament.